



**The University of Azad Jammu and Kashmir,
Muzaffarabad**

Name: Muneeba Arshad

Roll NO: 2024-SE-06

Semester:4th

Course Title: Database System

Course Code: CS-2204

Submitted To: Engr. Awais Rathore

Department of Software Engineering

Department Of Software Engineering

✓ **Aggregate Function**

1. Introduction

Aggregate functions are SQL functions that are used to summarize data from multiple rows and produce meaningful results. In this laboratory, the main aggregate functions COUNT, SUM, AVG, MIN and MAX were studied and implemented using MySQL.

The laboratory also focuses on GROUP BY and HAVING. GROUP BY is used to divide records into groups and perform aggregate calculations for each group. HAVING is used to filter groups after aggregation.

The practical work was performed using a RetailStore database containing Customer, Product and OrderItem tables. An additional University database was used for the assessment tasks.

2. Objectives

The main objectives of this laboratory are:

1. To understand aggregate functions in SQL.
2. To use COUNT, SUM, AVG, MIN and MAX.
3. To understand COUNT(*) and COUNT(column).
4. To use COUNT(DISTINCT column).
5. To understand NULL values during aggregation.
6. To use GROUP BY for grouping records.
7. To use HAVING for filtering groups.
8. To understand the difference between WHERE and HAVING.
9. To combine aggregate functions with JOIN operations.
10. To calculate useful summaries from database records.

3. Software and Tools

The following tools were used:

- XAMPP
- MySQL
- phpMyAdmin
- Web Browser
- Microsoft Word

The MySQL database was created and managed through phpMyAdmin.

4. Database Creation

The database was created in phpMyAdmin using the following SQL command:

```
CREATE DATABASE IF NOT EXISTS agg_lab;  
USE agg_lab;
```

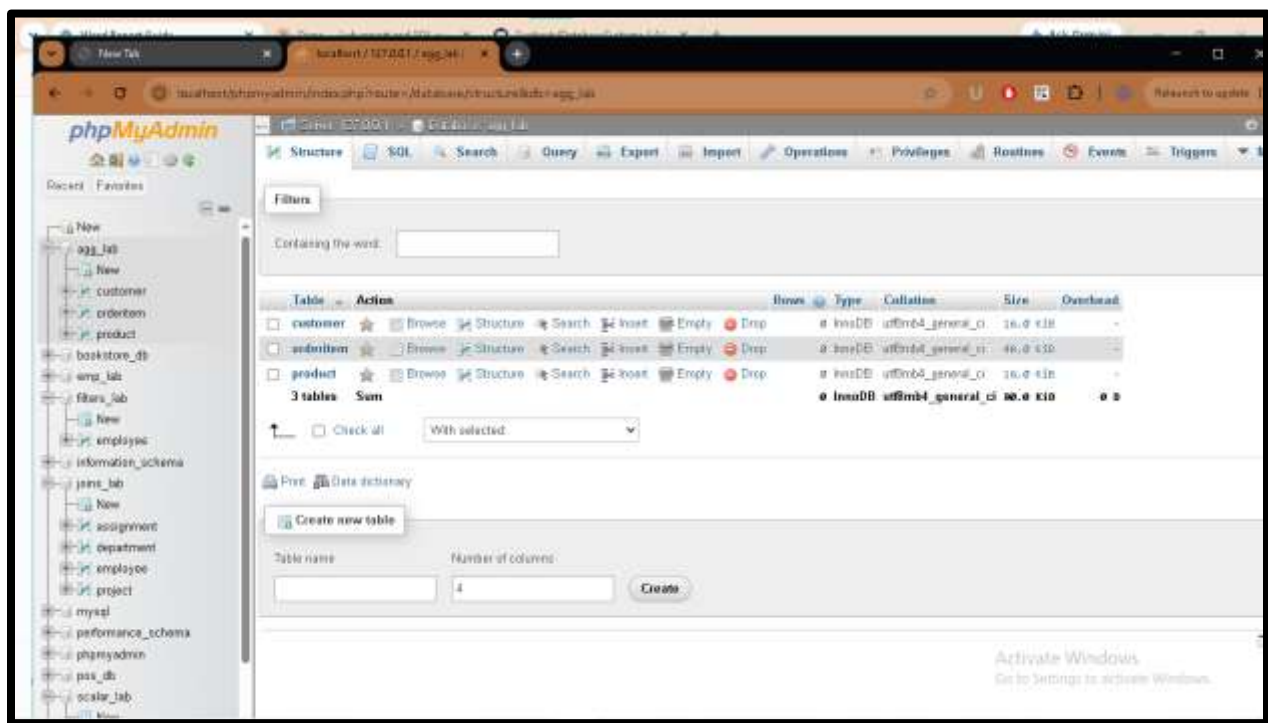
The Customer, Product and OrderItem tables were then created using primary keys and foreign keys.

6. Table Creation and Data Insertion

Three tables were created in the database:

- Customer
- Product
- OrderItem

Sample data was inserted into all three tables. The database contains 8 customers, 10 products and 15 order records.



7. Aggregate Functions

Aggregate functions operate on multiple rows and return a summarized value.

The main aggregate functions used in this laboratory are:

Function	Purpose
COUNT()	Counts rows or non-NULL values
SUM()	Calculates the total
AVG()	Calculates the average
MIN()	Finds the minimum value
MAX()	Finds the maximum value

COUNT(*) counts all rows, while COUNT(column) counts only non-NULL values.
COUNT(DISTINCT column) counts unique non-NULL values.

8. Part A – Whole Table Aggregates

A1. Total Number of Customers, Products and Orders

```
SELECT
  (SELECT COUNT(*) FROM Customer) AS TotalCustomers,
  (SELECT COUNT(*) FROM Product) AS TotalProducts,
  (SELECT COUNT(*) FROM OrderItem) AS TotalOrders;
```

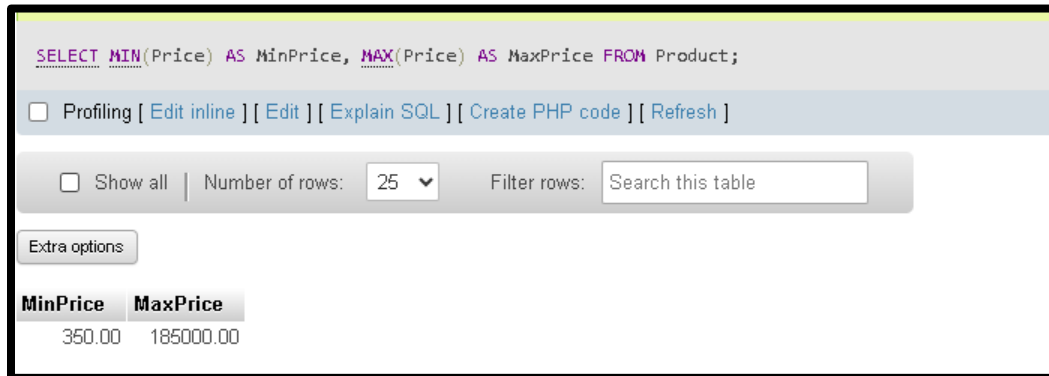
The result shows:

- Total Customers = 8
- Total Products = 10
- Total Orders = 15



A2. Cheapest and Most Expensive Product

```
SELECT
  MIN(Price) AS MinPrice,
  MAX(Price) AS MaxPrice
FROM Product;
```



The screenshot shows a SQL query execution interface. At the top, the query is displayed: `SELECT MIN(Price) AS MinPrice, MAX(Price) AS MaxPrice FROM Product;`. Below the query, there are options for Profiling, Edit inline, Edit, Explain SQL, Create PHP code, and Refresh. A control bar includes a 'Show all' checkbox, 'Number of rows' set to 25, and a 'Filter rows' search box. An 'Extra options' button is also present. The results are displayed in a table with two columns: **MinPrice** and **MaxPrice**. The values shown are 350.00 and 185000.00 respectively.

MinPrice	MaxPrice
350.00	185000.00

The query uses MIN() and MAX() to find the lowest and highest product prices.

A3. Average Product Price

```
SELECT ROUND(AVG(Price), 2) AS AveragePrice
FROM Product;
```



The screenshot shows a SQL query execution interface. The query is: `SELECT ROUND(AVG(Price), 2) AS AveragePrice FROM Product;`. The interface includes the same control bar as the previous screenshot. The results are displayed in a table with one column: **AveragePrice**. The value shown is 26265.05.

AveragePrice
26265.05

AVG() calculates the average price of all products.

A4. Total Stock Quantity

```
SELECT SUM(StockQty) AS TotalStock
FROM Product;
```

☐ Show all	Number of rows: 25	Filter rows: Search this table
Extra options		
TotalStock 618		
☐ Show all	Number of rows: 25	Filter rows: Search this table

SUM() is used to calculate the total stock quantity.

A5. Number of Distinct Cities

```
SELECT COUNT(DISTINCT City) AS DistinctCities
FROM Customer;
```

SELECT COUNT(DISTINCT City) AS DistinctCities FROM Customer;	
☐ Profiling	Edit inline Edit Explain SQL Create PHP code Refresh
Extra options	
DistinctCities 3	
Query results operations	

This query counts unique cities and ignores NULL values.

A6. Number of Distinct Product Categories

```
SELECT COUNT(DISTINCT Category) AS DistinctCategories
FROM Product;
```

Extra options	
DistinctCategories 4	
Query results operations	
Print	Copy to clipboard
Export	Display chart
Create view	

The query counts the unique product categories.

A7. Customers With and Without a Recorded City

Customers with city

```
SELECT COUNT(City) AS CustomersWithCity  
FROM Customer;
```

The screenshot shows a web-based SQL interface. At the top, there are links for 'Profiling', 'Edit inline', 'Edit', 'Explain SQL', 'Create PHP code', and 'Refresh'. Below these is a button for 'Extra options'. The query result is displayed in a table with one row and one column, showing the value '7'. The column header is 'CustomersWithCity'. At the bottom, there is a section for 'Query results operations' with buttons for 'Print', 'Copy to clipboard', 'Export', 'Display chart', and 'Create view'.

Customers without city

```
SELECT COUNT(*) - COUNT(City) AS CustomersWithoutCity  
FROM Customer;
```

The screenshot shows a web-based SQL interface. At the top, there is a yellow message box that says 'Your SQL query has been executed successfully.' Below this is the SQL query: 'SELECT COUNT(*) - COUNT(City) AS CustomersWithoutCity FROM Customer;'. There are links for 'Profiling', 'Edit inline', 'Edit', 'Explain SQL', 'Create PHP code', and 'Refresh'. Below these is a button for 'Extra options'. The query result is displayed in a table with one row and one column, showing the value '1'. The column header is 'CustomersWithoutCity'. At the bottom, there is a section for 'Query results operations' with buttons for 'Print', 'Copy to clipboard', 'Export', 'Display chart', and 'Create view'.

COUNT(City) ignores the NULL city value.

A8. Earliest and Latest Order Date

```
SELECT  
    MIN(OrderDate) AS EarliestOrder,  
    MAX(OrderDate) AS LatestOrder  
FROM OrderItem;
```

☐ Show all	Number of rows: 25	Filter rows: Search this table
Extra options		
EarliestOrder	LatestOrder	
2023-03-10	2024-10-11	
☐ Show all	Number of rows: 25	Filter rows: Search this table
Query results operations		

MIN() finds the earliest date and MAX() finds the latest date.

A9. Total Revenue

Revenue is calculated using:

Quantity × Price

```
SELECT
  SUM(o.Quantity * p.Price) AS TotalRevenue
FROM OrderItem o
JOIN Product p
ON o.ProdID = p.ProdID;
```

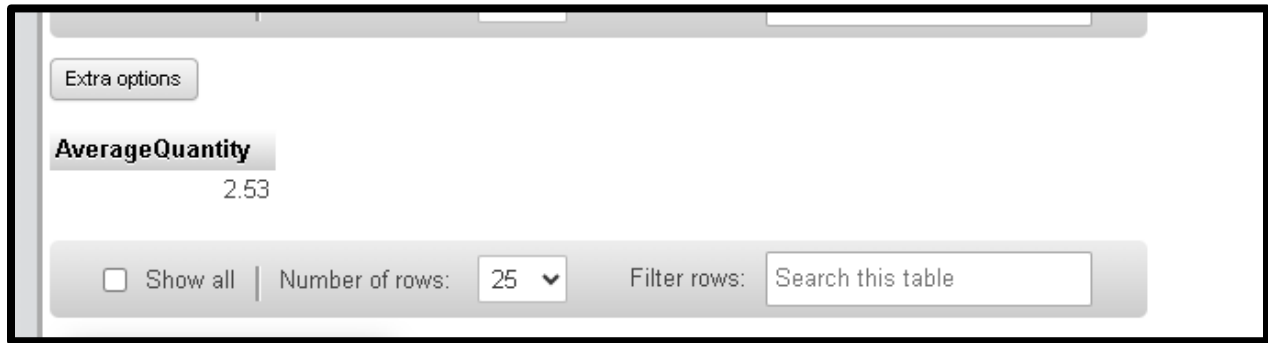
☐ Show all	Number of rows: 25	Filter rows: Search this table
Extra options		
TotalRevenue		
476800.45		
☐ Show all	Number of rows: 25	Filter rows: Search this table

The Product and OrderItem tables are joined to calculate the total revenue.

A10. Average Quantity Per Order

```
SELECT ROUND(AVG(Quantity), 2) AS AverageQuantity
```

FROM OrderItem;



AverageQuantity
2.53

AVG() calculates the average quantity of products in an order record.

8. GROUP BY

GROUP BY is used to divide records into groups. Aggregate functions can then be applied separately to every group.

For example, customers can be grouped according to their city.

```
SELECT City, COUNT(*) AS NumCustomers
FROM Customer
GROUP BY City;
```

GROUP BY produces one result row for every group.

10. HAVING

HAVING is used to filter groups after aggregation.

WHERE filters individual rows before grouping, while HAVING filters groups after GROUP BY.

Example:

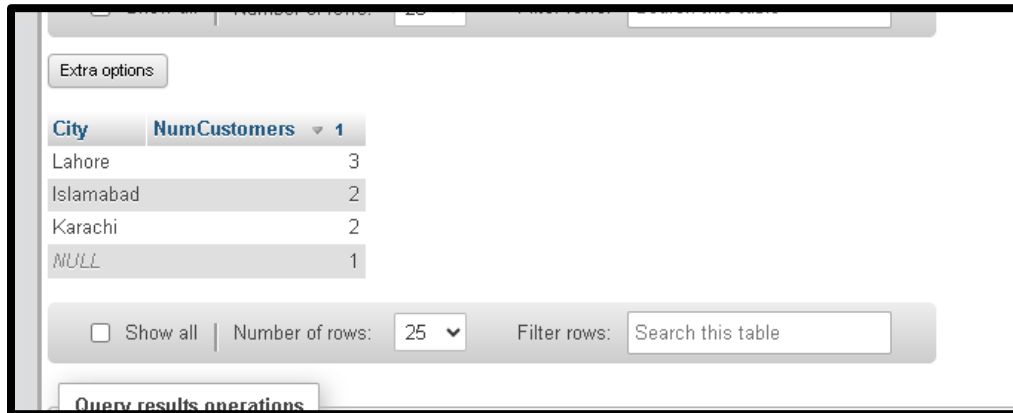
```
SELECT
    Category,
    AVG(Price) AS AvgPrice
FROM Product
GROUP BY Category
HAVING AVG(Price) > 5000;
```

HAVING is useful when the condition contains an aggregate function such as COUNT(), SUM() or AVG().

11. Part B – GROUP BY, HAVING and JOINS

B1. Number of Customers in Each City

```
SELECT City, COUNT(*) AS NumCustomers
FROM Customer
GROUP BY City
ORDER BY NumCustomers DESC;
```



City	NumCustomers
Lahore	3
Islamabad	2
Karachi	2
NULL	1

B2. Number of Products in Each Category

B3. Average, Minimum and Maximum Price Per Category

```
SELECT
    Category,
    ROUND(AVG(Price), 2) AS AvgPrice,
    MIN(Price) AS MinPrice,
    MAX(Price) AS MaxPrice
FROM Product
GROUP BY Category
ORDER BY AvgPrice DESC;
```

Extra options

				Category	AvgPrice	MinPrice	MaxPrice
☐	Edit	Copy	Delete	Electronics	48950.00	800.00	185000.00
☐	Edit	Copy	Delete	Furniture	31750.25	18500.00	45000.50
☐	Edit	Copy	Delete	Grocery	1275.00	650.00	1899.99
☐	Edit	Copy	Delete	Stationery	400.00	350.00	450.00

☐ Check all With selected: Edit Copy Delete Export

☐ Show all Number of rows: 25 Filter rows: Search this table

B4. Total Stock Quantity Per Category

SELECT

Category,

SUM(StockQty) AS TotalStock

FROM Product

GROUP BY Category

ORDER BY TotalStock DESC;

☐ Show all Number of rows: 25 Filter rows: Search this table

Extra options

				Category	TotalStock
☐	Edit	Copy	Delete	Stationery	360
☐	Edit	Copy	Delete	Electronics	180
☐	Edit	Copy	Delete	Grocery	75
☐	Edit	Copy	Delete	Furniture	13

☐ Check all With selected: Edit Copy Delete Export

B5. Number of Orders Per Year

SELECT

YEAR(OrderDate) AS OrderYear,

COUNT(*) AS NumOrders

FROM OrderItem

GROUP BY YEAR(OrderDate)

ORDER BY OrderYear;

Extra options	
OrderYear ▲ 1	NumOrders
2023	7
2024	8
☐ Show all Number of rows: 25 ▼ Filter rows: Search this table	

B6. Number of Orders Per Month in 2024

SELECT

MONTH(OrderDate) AS Month,

COUNT(*) AS NumOrders

FROM OrderItem

WHERE YEAR(OrderDate) = 2024

GROUP BY MONTH(OrderDate)

ORDER BY Month;

☐ Show all Number of rows: 25 ▼ Filter rows: Search this table	
Extra options	
Month ▲ 1	NumOrders
1	2
2	1
4	2
7	1
9	1
10	1
☐ Show all Number of rows: 25 ▼ Filter rows: Search this table	

B7. Categories With Average Price Greater Than 5,000

SELECT

Category,

ROUND(AVG(Price), 2) AS AvgPrice

FROM Product
 GROUP BY Category
 HAVING AVG(Price) > 5000;

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

				Category	AvgPrice
☐	Edit	Copy	Delete	Electronics	48950.00
☐	Edit	Copy	Delete	Furniture	31750.25

☐ Check all | With selected: Edit Copy Delete Export

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

B8. Cities With More Than One Customer

SELECT
 City,
 COUNT(*) AS NumCustomers
 FROM Customer
 WHERE City IS NOT NULL
 GROUP BY City
 HAVING COUNT(*) > 1
 ORDER BY NumCustomers DESC;

Extra options

City	NumCustomers
Lahore	3
Karachi	2
Islamabad	2

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Query results operations

B9. Number of Orders Per Customer

Customers with zero orders are also included.

```

SELECT
    c.CustName,
    COUNT(o.OrderID) AS NumOrders
FROM Customer c
LEFT JOIN OrderItem o
ON c.CustID = o.CustID
GROUP BY c.CustID, c.CustName

```

CustName	NumOrders
Ali Khan	2
Sara Iqbal	3
Hamza Raza	3
Ayesha Noor	1
Bilal Ahmed	2
Fatima Sheikh	1
Usman Tariq	3
Maira Javed	0

```
ORDER BY NumOrders DESC;
```

LEFT JOIN is used because customers without orders must also appear.

B10. Total Quantity Sold Per Product

```

SELECT
    p.ProdName,
    COALESCE(SUM(o.Quantity), 0) AS TotalQty
FROM Product p
LEFT JOIN OrderItem o
ON p.ProdID = o.ProdID
GROUP BY p.ProdID, p.ProdName
ORDER BY TotalQty DESC;

```

Extra options	
ProdName	TotalQty ▾ 1
Notebook A4	15
Coffee Beans 1kg	5
USB-C Cable	4
Green Tea Box	3
Wireless Mouse	3
Ballpoint Pen 10pk	2
Bluetooth Speaker	2
Laptop Pro 15	2
Office Chair	1
Standing Desk	1

☐ Show all | Number of rows: 25 ▾ | Filter rows: | Sort by key: None ▾

B11. Total Revenue Per Product Category

SELECT

p.Category,

SUM(o.Quantity * p.Price) AS Revenue

FROM OrderItem o

JOIN Product p

ON o.ProdID = p.ProdID

GROUP BY p.Category

ORDER BY Revenue DESC;

☐ Show all Number of rows: 25 ▾ Filter rows: Search this table	
Extra options	
Category	Revenue ▾ 1
Electronics	395700.00
Furniture	63500.50
Grocery	11449.95
Stationery	6150.00

☐ Show all | Number of rows: 25 ▾ | Filter rows:

B12. Total Spend Per Customer

SELECT

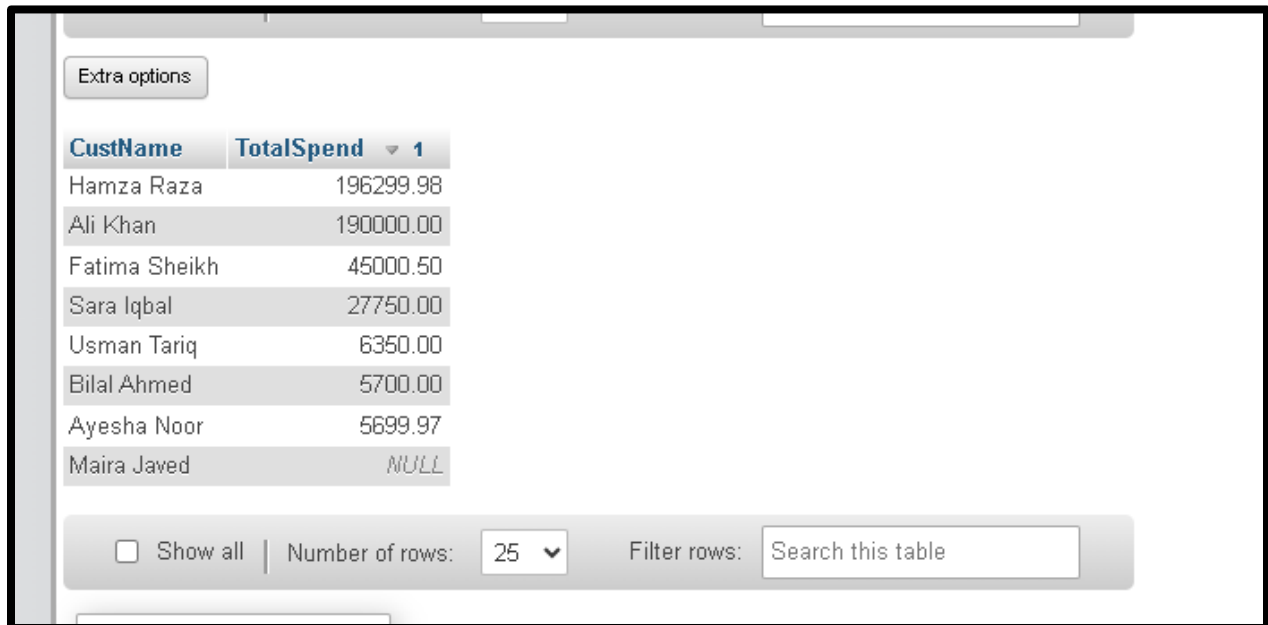
c.CustName,

SUM(o.Quantity * p.Price) AS TotalSpend

```

FROM Customer c
LEFT JOIN OrderItem o
ON c.CustID = o.CustID
LEFT JOIN Product p
ON o.ProdID = p.ProdID
GROUP BY c.CustID, c.CustName
ORDER BY TotalSpend DESC;

```



The screenshot shows a web-based interface for displaying query results. At the top, there is a button labeled "Extra options". Below it is a table with two columns: "CustName" and "TotalSpend". The table contains eight rows of data, with the last row showing a NULL value for TotalSpend. At the bottom of the interface, there are controls for displaying the results: a checkbox for "Show all", a "Number of rows:" dropdown set to "25", and a "Filter rows:" search box with the placeholder text "Search this table".

CustName	TotalSpend
Hamza Raza	196299.98
Ali Khan	190000.00
Fatima Sheikh	45000.50
Sara Iqbal	27750.00
Usman Tariq	6350.00
Bilal Ahmed	5700.00
Ayesha Noor	5699.97
Maira Javed	NULL

B13. Customers With Total Spend Above 50,000

```

SELECT
    c.CustName,
    SUM(o.Quantity * p.Price) AS TotalSpend
FROM Customer c
JOIN OrderItem o
ON c.CustID = o.CustID
JOIN Product p
ON o.ProdID = p.ProdID
GROUP BY c.CustID, c.CustName
HAVING SUM(o.Quantity * p.Price) > 50000
ORDER BY TotalSpend DESC;

```

☐ Show all	Number of rows: 25	Filter rows: Search this table
Extra options		
CustName	TotalSpend	▼ 1
Hamza Raza	196299.98	
Ali Khan	190000.00	
☐ Show all	Number of rows: 25	Filter rows: Search this table
Query results operations		

B14. Customers and Revenue Per City

SELECT

c.City,

COUNT(DISTINCT c.CustID) AS NumCustomers,

SUM(o.Quantity * p.Price) AS TotalRevenue

FROM Customer c

JOIN OrderItem o

ON c.CustID = o.CustID

JOIN Product p

ON o.ProdID = p.ProdID

GROUP BY c.City

HAVING COUNT(DISTINCT c.CustID) > 1

ORDER BY TotalRevenue DESC;

Extra options		
City	NumCustomers	TotalRevenue ▼ 1
Lahore	3	392649.98
Karachi	2	33450.00
☐ Show all	Number of rows: 25	Filter rows: Search this table

B15. Top Three Best-Selling Products

SELECT

p.ProdName,

SUM(o.Quantity) AS TotalQty

FROM Product p

```

JOIN OrderItem o
ON p.ProdID = o.ProdID
GROUP BY p.ProdID, p.ProdName
ORDER BY TotalQty DESC
LIMIT 3;

```

Extra options	
ProdName	TotalQty ▾ 1
Notebook A4	15
Coffee Beans 1 kg	5
USB-C Cable	4
Query results operations	

B16. Total Revenue Per Year

```

SELECT
    YEAR(o.OrderDate) AS OrderYear,
    SUM(o.Quantity * p.Price) AS Revenue
FROM OrderItem o
JOIN Product p
ON o.ProdID = p.ProdID
GROUP BY YEAR(o.OrderDate)
ORDER BY OrderYear;

```

☐ Show all	Number of rows: 25 ▾	Filter rows: Search this table
Extra options		
OrderYear ▴ 1	Revenue	
2023	408449.97	
2024	68350.48	
☐ Show all	Number of rows: 25 ▾	Filter rows: Search this table
Query results operations		

B17. Average Order Value

```

SELECT
    ROUND(AVG(OrderTotal), 2) AS AverageOrderValue
FROM (
    SELECT
        o.OrderID,

```

```
SUM(o.Quantity * p.Price) AS OrderTotal
FROM OrderItem o
JOIN Product p
ON o.ProdID = p.ProdID
GROUP BY o.OrderID
) AS OrderTotals;
```

Extra options
AverageOrderValue
31786.70
☐ Show all Number of rows: 25 v Filter rows: Search this table
Query results operations